



Introduction to NumPy

NumPy is a high-performance library in Python used for numerical operations. It is particularly useful when dealing with large datasets or matrices. It offers better performance than Python lists due to fixed types and memory efficiency.

```
import numpy as np
```

Array Creation

You can create arrays from lists or use built-in functions like zeros, ones, arange, etc.

```
a = np.array([10, 20, 30])
b = np.zeros((2, 2))
c = np.ones((3, 1))
d = np.arange(0, 10, 2)
e = np.linspace(1, 5, 4)

print(a) # [10 20 30]
print(b) # [[0. 0.] [0. 0.]]
print(c) # [[1.] [1.] [1.]]
print(d) # [0 2 4 6 8]
print(e) # [1. 2.333 3.666 5.]
```

Indexing and Slicing

Accessing elements is similar to Python lists but works on multi-dimensional arrays too.

```
arr = np.array([[5, 10, 15], [20, 25, 30]])
print(arr[0][2]) # 15
print(arr[1, :2]) # [20 25]
print(arr[:, 1]) # [10 25]
```



Shape and Reshape

NumPy arrays have a shape attribute to show dimensions and can be reshaped.

```
arr = np.arange(12)
print(arr.shape) # (12,)
arr2d = arr.reshape(3, 4)
print(arr2d.shape) # (3, 4)
print(arr2d)
```

Data Types (dtype)

Arrays have fixed types. You can check or change them.

```
arr = np.array([1.5, 2.5, 3.5])
print(arr.dtype) # float64
new_arr = arr.astype(int)
print(new_arr) # [1 2 3]
```

Iterating Over Arrays

```
arr = np.array([[1, 2], [3, 4]])
for row in arr:
    for item in row:
        print(item, end=' ') # 1 2 3 4
```

Mathematical Operations

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
print(a + b) # [11 22 33]
print(a * 2) # [20 40 60]
print(np.sqrt(a)) # [3.16 4.47 5.47]
```

Broadcasting

```
arr = np.array([[1, 2], [3, 4]])  
print(arr + 10)  
# [[11 12]  
#    [13 14]]
```

Broadcasting allows arithmetic between arrays of different shapes.

Aggregations

```
arr = np.array([[1, 2], [3, 4]])  
print(np.sum(arr))      # 10  
print(np.mean(arr))     # 2.5  
print(np.min(arr, axis=0)) # [1 2]
```

Comparisons and Conditions

```
arr = np.array([5, 10, 15, 20])  
print(arr > 10)          # [False False  True  True]  
print(np.where(arr > 10, 1, 0)) # [0 0 1 1]
```

Copy vs View

```
arr = np.array([1, 2, 3])  
copy_arr = arr.copy()  
view_arr = arr.view()  
arr[0] = 99  
print(copy_arr) # [1 2 3]  
print(view_arr) # [99 2 3]
```

Handling NaN and Infinity

```
arr = np.array([1, np.nan, np.inf])  
print(np.isnan(arr)) # [False True False]
```



```
print(np.isinf(arr)) # [False False True]
print(np.nan_to_num(arr)) # [1.  0. 1.797e+308]
```

Sorting and Searching

```
arr = np.array([30, 10, 20])
print(np.sort(arr)) # [10 20 30]
print(np.argsort(arr)) # [1 2 0]
print(np.where(arr > 15)) # (array([0, 2]),)
```

Matrix Operations

```
mat = np.array([[1, 2], [3, 4]])
print(mat.T) # Transpose
print(np.linalg.inv(mat)) # Inverse
print(np.dot(mat, mat)) # Matrix multiplication
```

Practical Use Cases

- Data cleaning: fill NaN, filter rows
- Stats & probability: mean, std, simulations
- Matrix algebra: useful in ML
- Fast math computation in finance or engineering

Practice Questions

1. Analyze daily temperatures:

```
temps = np.array([32.5, 33.1, 34.8, 36.2, 35.9, 34.0, 33.5])
```

Find the highest and lowest temperature.

2. Student marks for 5 students in 3 subjects:

```
marks = np.array([[78, 85, 90], [88, 76, 92], [67, 80, 85], [90, 95, 88],  
[76, 70, 82]])
```

Calculate percentage for each student and assign grades:

A (≥ 85),

B ($70-84$),

C (< 70)

3. Monthly electricity usage (in units) of 4 households:

```
usage = np.array([[120, 130, 110, 140, 150, 145, 160, 170, 155, 150,  
160, 165], [100, 110, 105, 120, 130, 125, 140, 135, 130, 125, 130, 120],  
[90, 95, 100, 110, 115, 120, 125, 130, 135, 140, 145, 150], [200, 190,  
210, 220, 215, 205, 200, 195, 210, 220, 230, 240]])
```

Find yearly total units consumed for each household.

4. Product ratings from 100 users:

```
ratings = np.random.randint(1, 6, 100)
```

Find the most common rating.

Normalize this age data using Min-Max scaling:

```
ages = np.array([18, 25, 32, 45, 60, 75])
```

Formula: $(\text{value} - \text{min}) / (\text{max} - \text{min})$